Agentry

Beyond Static DAGs: Orchestrating AI-Generated Multi-Agent Workflows

Cong Wang

Founder and CEO, Agentry, Inc.

AGNTCon North America 2026

Agentry

The New Business Automation

Today: inside one organization

- Coding, support, and research agents
- One company, one stack, one trust boundary
- A framework is enough

Tomorrow: across organizations

- Part X is short: a planner agent decides who to ask, at runtime
- Every hop crosses a company, a stack, and a trust boundary

> *Soon, agents will do business with other companies' agents.*

Agentry

Business at Agent Speed

Part X is short. Your planner agent writes this plan on the fly:

```
{ "goal": "Part X is short 5,000 units",
  "plan": [
    { "parallel": ["quotes@supplier-a.com", "quotes@supplier-b.com", "stock@maker.com"] },
    { "ask": "buyer@maker.com", "if": "within_budget == true",
      "then": "purchasing@maker.com", "else": "approver@maker.com" } ] }
```

Now what runs it?

- Who fans out, who waits, and what if `supplier-b` never answers?
- Who checks that a quote is real?
- Each supplier built its agents on a different stack. Now what?

Agentry

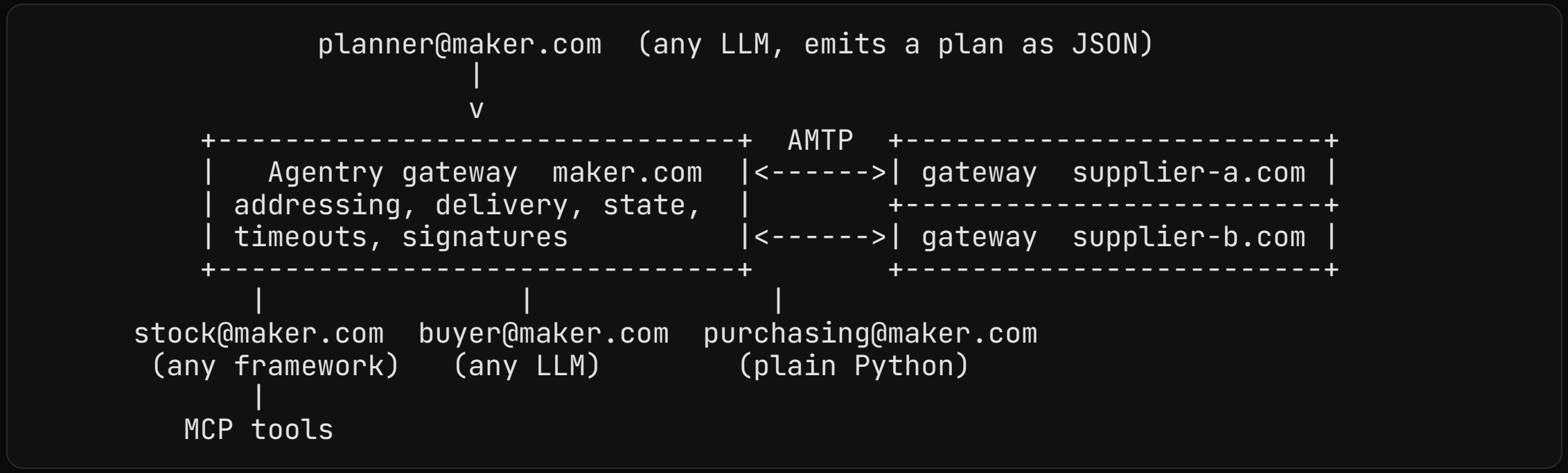
Every Layer Says "Not My Job"

Tool	Orchestrates	Why it does not run this plan
Kubernetes	Containers	No notion of an agent, a reply, or a workflow
Kubeflow / Argo	Container pipelines	Every step is a pod in your cluster, not an agent you address
LangGraph / CrewAI	Agents inside one app	Shared state ends at the app, A2A calls carry no workflow state
Temporal	Workflows as code	You write the interpreter, no agent addressing across orgs
A2A	One agent calling another	No multi-step plan across many agents
MCP	Agent to tool	No multi-agent coordination

> *Nothing runs a generated plan across agents and orgs.*

Agentry

Where Agentry Sits



Frameworks build the agent. MCP gives it tools. **Agentry runs the work between agents.**

Agentry

Everything Is a Message

```
{
  "sender":       "planner@maker.com",                <- Part 1: address,
  "recipients":   ["quotes@supplier-a.com", ...],        deliver, and
  "payload":      { "part": "X", "quantity": 5000 },     sign the message
  "coordination": { "type": "parallel", ... }         <- Part 2: run it as
}                                                        a workflow
```

- The plan, each step, and every reply travel as messages
- Drop `coordination` and it is plain agent mail
- First the message, then the workflow built on it

PART 1

AMTP: Email for Agents

Addressing, delivery, and trust across organizations

Agentry

Why Email?

Email already connects any two organizations. AMTP borrows its design:

Email	AMTP
`alice@maker.com`	`quotes@supplier-a.com`
MX record	`_amtp.<domain>` TXT record points to the gateway
Store and forward, retries, bounces	At-least-once delivery, idempotency, status API
DKIM	Domain signatures, keys in `<selector>._amtpkey.<domain>`
No central registry	Each org runs its own gateway, no shared control plane

Email lacks two things agents need: schema-validated payloads and coordination.

Agentry

The Gateway Owns Delivery

- **At-least-once:** UUIDv7 `message_id` plus `idempotency_key`, duplicates dropped
- **Retries:** exponential backoff, ±25% jitter, 5 minute cap, retryable errors only
- **Status:** `GET /v1/messages/{id}/status`, per recipient
- **Inbox:** push to the agent's webhook, or let it pull from `GET /v1/inbox/{address}`
- **Schemas:** AGNTCY schema IDs, payloads validated at the gateway
- **Deploy:** single binary, Docker, Helm chart, PostgreSQL for durable storage

Your agent code handles one message and sends one reply. The gateway handles the network.

Agentry

Trust Across Organizations

Domain signatures, DKIM for agents:

- Sender's gateway signs the body: RFC 8785 JSON, ES256 or RS256
- Receiver fetches `<selector>._amtpkey.<sender domain>`, verifies the raw bytes
- Gateway stamps `verified` or `invalid` itself, never trusts the wire
- Policy for unsigned remote mail: `accept`, `flag`, or `reject`
- Local agents authenticate with a per-agent API key
- Delivered is not trusted: an unsigned reply never advances a workflow

> *Suppliers sign their quotes. A forged quote is ignored.*

PART 2

Running the Plan

The workflow is data, not code

Agentry

Three Coordination Primitives

A workflow is a `coordination` block on an ordinary message:

Type	Fields	Semantics
`parallel`	`required_responses`, `optional_responses`	Fan out to all, finish when every participant has answered
`sequential`	`sequence`	One agent at a time, next step dispatched on each reply
`conditional`	`recipients`, `conditions: [{if, then, else}]`	Evaluate the first reply, run one branch, mark the other skipped

Common to all: `timeout` and `stop_on_failure`.

The planner emits JSON, the gateway runs it.

Agentry

From Plan to Message

```
{
  "sender": "planner@maker.com",
  "recipients": ["quotes@supplier-a.com", "quotes@supplier-b.com", "stock@maker.com"],
  "subject": "Quote request: part X",
  "coordination": {
    "type": "parallel",
    "timeout": 86400,
    "required_responses": ["quotes@supplier-a.com", "quotes@supplier-b.com",
                           "stock@maker.com"],
    "stop_on_failure": false
  },
  "payload": { "part": "X", "quantity": 5000, "needed_by": "2026-11-20" }
}
```

One `POST /v1/messages` returns a `workflow_id`. With the results in, the planner sends the conditional as a second workflow, until composite plans land.

Agentry

The Workflow Lifecycle

```
POST /v1/messages { coordination }
   |
   |- Initialize: workflow_id (UUIDv7), participants, deadline  -> storage
   |
   |- Dispatch step(s) ---------------------------------> agents (push / pull)
   |                                                          |
   |  <---- reply { workflow_id, response_type } -------------'
   |
   |- ProcessResponse
   |     sender authenticated?  participant still pending?
   |     compare-and-swap on workflow version
   |     evaluate: next in sequence | take branch | done
   |
   '- workflow@maker.com --> planner@maker.com : aggregated results
```

Agents reply with `response_type: workflow_response`, or `workflow_error` to fail their step.

Agentry

When Things Go Wrong

What goes wrong	What Agentry does
An agent never answers	Deadline sweeper marks pending steps `timeout`, notifies the sender
An agent fails	`workflow_error` reply, `stop_on_failure` ends the workflow early
A step is re-dispatched	Idempotency key = hash(workflow, step), receivers deduplicate
Two replies race on two replicas	Optimistic version check on shared PostgreSQL, loser retries
Duplicate or forged reply	Dropped: participant not pending, or sender not authenticated
Network blip	Per-message retries with exponential backoff

> *State lives in the gateway, not in the planner's context window.*

Agentry

Conditionals

```
"recipients": ["buyer@maker.com"],
"coordination": { "type": "conditional", "timeout": 3600,
  "conditions": [ { "if": "within_budget == true",
                    "then": ["purchasing@maker.com"],
                    "else": ["approver@maker.com"] } ] }
```

- `buyer@` compares the quotes and replies, the gateway evaluates `if` against its payload
- Matching branch is dispatched, the other is marked `skipped`
- Today: `key == value` on the reply payload
- Next: numeric comparisons, such as `total_price < 50000`

Agentry

Shared Context IN PROGRESS

Today: replies are stored per step, the sender gets them all at the end:

```
{ "workflow_id": "0199...", "coordination_type": "parallel", "status": "timeout",
  "results": [
    { "address": "quotes@supplier-a.com", "status": "completed", "payload": { } },
    { "address": "quotes@supplier-b.com", "status": "timeout",   "payload": null } ] }
```

Next:

- Context shared between agents in a workflow: `buyer@` sees every quote, no wiring
- Token-aware selection and summarization of what gets forwarded
- Composite plans: parallel, then conditional, as one workflow

PART 3

Fitting Your Stack

Your agents, your framework, and Agentry

Agentry

Adding an Agent

Register once: `POST /v1/admin/agents {"address": "stock", "delivery_mode": "push", "push_target": ...}`

```python
@app.post("/webhook")                          # Agentry pushes here
def on_message(msg: dict):
    result = check_inventory(msg["payload"])         # any framework, or plain code
    requests.post(f"{GATEWAY}/v1/messages",
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={
            "sender": "stock@maker.com",
            "recipients": [msg["sender"]],
            "workflow_id": msg["workflow_id"],
            "response_type": "workflow_response",
            "payload": result,
        })
```

Plain HTTP shown, so any language works. Python SDK: `pip install amtp`

Agentry

Engine or Framework?

Reach for an orchestration engine when:

- Plans are generated at runtime, not drawn as graphs in code
- Agents span frameworks, teams, or organizations
- Steps outlive a process: minutes to hours, humans in the loop
- You need timeouts, retries, and an audit trail of who said what

Stay with your framework when:

- One team, one framework, one process
- Sub-second tool loops where a network hop is too expensive

Agentry

Today and Next

Works today

- AMTP gateway, DNS discovery
- Domain signatures
- Parallel, sequential, conditional
- Timeouts, failures, idempotent steps
- Conditions: `key == value`
- Results returned to the sender

Coming next

- Context shared between agents
- Context Graph
- Composite plans
- WASM plugins

Agentry

Key Takeaways

- AI planners produce workflows at runtime, static DAG engines cannot run them
- Treat the workflow as **data on a message**: parallel, sequential, conditional
- Borrow from email: `agent@domain`, DNS discovery, store and forward, signed domains
- Keep state, timeouts, and retries in the gateway, not in a context window
- Next frontier: context shared between agents automatically

> *Frameworks build agents. Protocols connect them. Agentry runs the work between them.*

Agentry

Questions?

Gateway: github.com/amtp-protocol/agentry
Protocol: github.com/amtp-protocol/amtp
Contact: cwang@agentry.site

Apache 2.0, contributions welcome.